



Instituto Tecnológico De Costa Rica

Escuela de Ingeniería en Computación

Diseño de software GR 20

Laboratorio 1: Patrones de Diseño Creacionales

Estudiante:

Luis Daniel Barboza Alfaro 2024211470

Profesor:

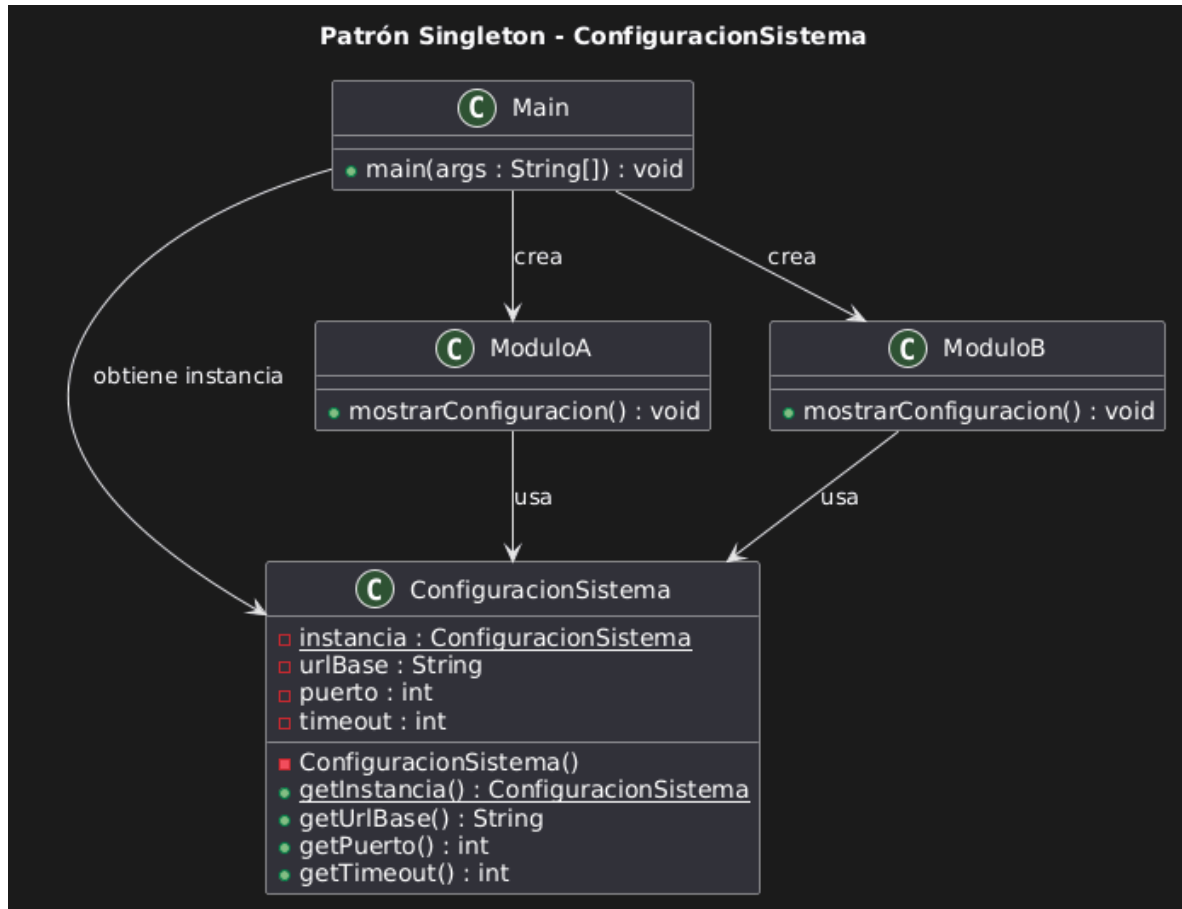
Carlos Guillermo Arias Rodriguez

I Semestre

2026

Singleton

Diagrama de Clases UML



Justificación del Patrón: ¿Por qué este patrón mejora la mantenibilidad en comparación con una solución procedimental?

El patrón Singleton mejora la mantenibilidad porque centraliza en un único lugar la creación y el acceso a la configuración del sistema. En una solución procedimental, cada módulo que necesitara la configuración tendría que crear su propio objeto **ConfiguracionSistema**, lo que significa que, si el puerto o la URL cambian, habría que buscar y modificar cada instancia por separado en todo el código.

Con Singleton, ese problema desaparece: hay un solo objeto en memoria y cualquier cambio en la configuración se refleja automáticamente en todos los módulos que la usan, sin tocar ninguna otra clase.

Adicionalmente, una solución procedimental no garantiza consistencia — dos módulos podrían tener valores distintos si cada uno cargó la configuración en

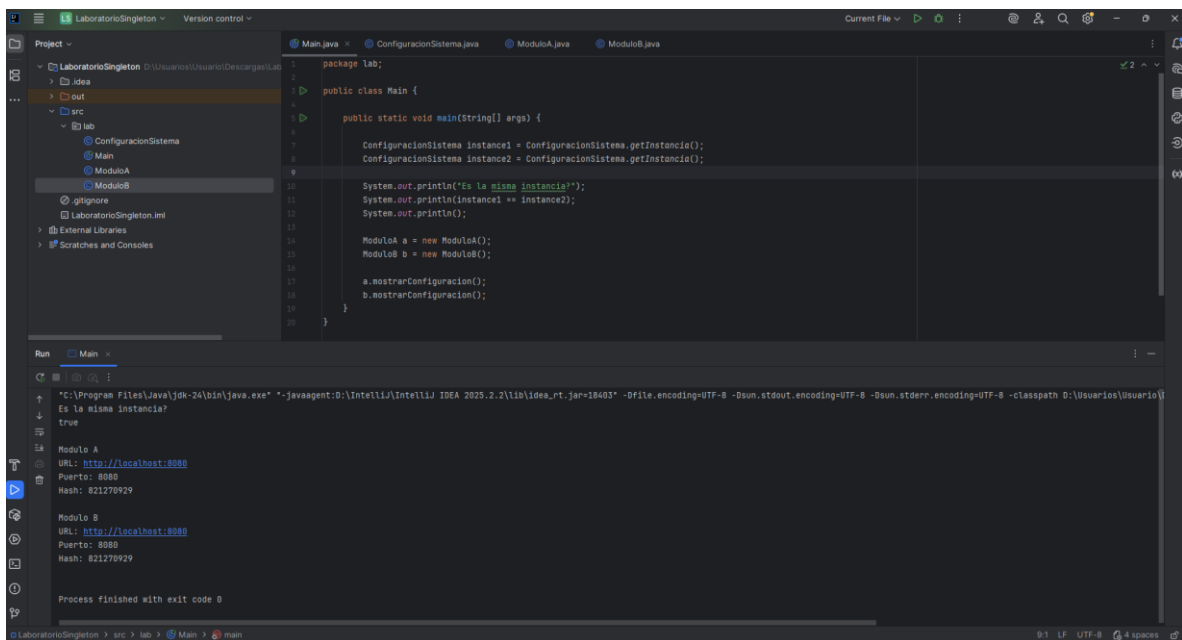
momentos diferentes. El Singleton elimina esa posibilidad por diseño, ya que el constructor es privado y la instancia se crea una sola vez.

Viéndolo más resumido, el uso de Singleton mejora la mantenibilidad al:

- Evitar duplicación de datos y lógica.
- Garantizar consistencia en la configuración.
- Centralizar cambios en un solo punto.
- Reducir el acoplamiento entre clases.

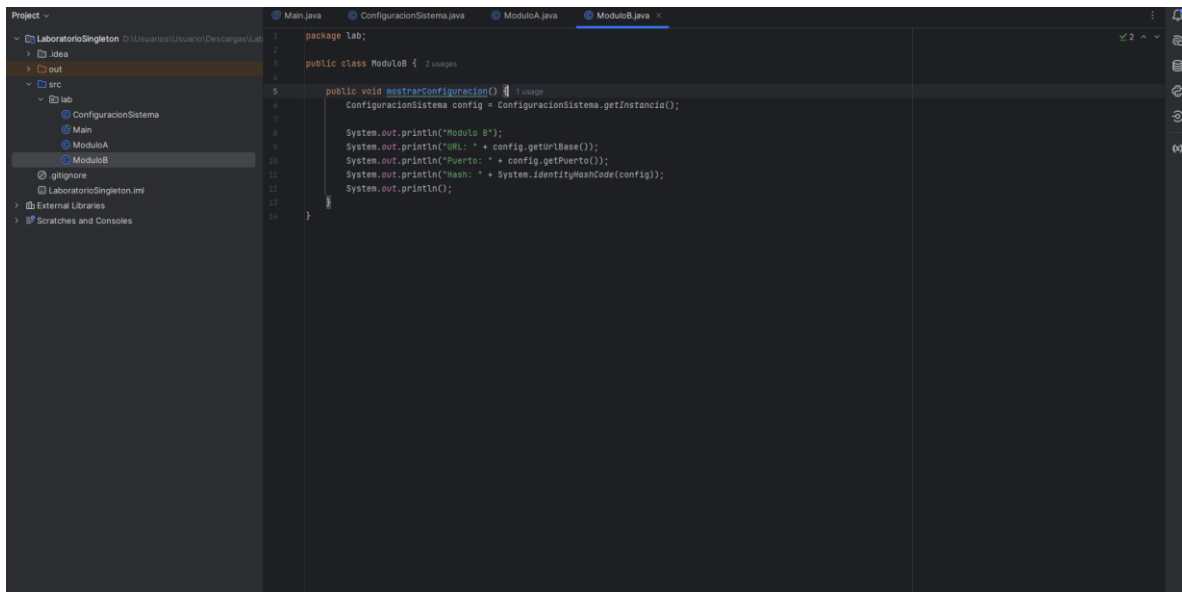
Esto lo hace una solución más robusta y escalable frente a un enfoque procedimental.

Ejecución del código en el IDE IntelliJ



```
1 package lab;
2
3 public class ConfiguracionSistema {
4
5     private static ConfiguracionSistema instancia;
6
7     private String urlBase;
8     private int puerto;
9     private int timeout;
10
11     private ConfiguracionSistema() {
12         this.urlBase = "http://localhost:8080";
13         this.puerto = 8080;
14         this.timeout = 30;
15     }
16
17     public static synchronized ConfiguracionSistema getInstance() {
18         if (instancia == null) {
19             instancia = new ConfiguracionSistema();
20         }
21         return instancia;
22     }
23
24     public String getUrlBase() {
25         return urlBase;
26     }
27
28     public int getPuerto() {
29         return puerto;
30     }
31
32     public int getTimeout() {
33         return timeout;
34     }
35 }
```

```
1 package lab;
2
3 public class ModuloA {
4
5     public void mostrarConfiguracion() {
6         ConfiguracionSistema config = ConfiguracionSistema.getInstance();
7
8         System.out.println("Modulo A");
9         System.out.println("URL: " + config.getUrlBase());
10        System.out.println("Puerto: " + config.getPuerto());
11        System.out.println("Hash: " + System.identityHashCode(config));
12        System.out.println();
13    }
14 }
```



Enlace al repositorio público en Github:

<https://github.com/DanielBA05/LabSingleton.git>